THE COMPLETE GUIDE TO

Object-Oriented Programming

in Python

Theory and practical implementation of every core concept,
with runnable code examples and their real outputs.

Classes & Objects | Encapsulation | Inheritance | Polymorphism
Abstraction | Magic Methods | Dataclasses | SOLID | Mini Project

20 chapters - 34 tested programs - Interview questions - Cheat sheet

Table of Contents

#	Chapter
1	Introduction to Object-Oriented Programming
2	Classes and Objects
3	Constructor (__init__) and the self Parameter
4	Instance Variables vs Class Variables
5	Types of Methods: Instance, Class and Static
6	Encapsulation and Access Modifiers
7	Properties: the Pythonic Getter and Setter
8	Inheritance
9	super() and Method Resolution Order (MRO)
10	Polymorphism
11	Method Overloading and Operator Overloading
12	Abstraction and Abstract Base Classes
13	Magic (Dunder) Methods
14	Composition and Aggregation (HAS-A Relationship)
15	Data Classes
16	Iterators, Generators and Context Managers with Classes
17	Custom Exceptions
18	Advanced Topics: __slots__, Class Decorators and Metaclasses
19	SOLID Design Principles
20	Mini Project: Library Management System
21	Interview Questions and Answers
22	Quick Reference Cheat Sheet

How to use this guide: each chapter starts with the theory, followed by key points and practical programs. Every program in this PDF was executed with Python 3, and the **Output** block shown beneath it is the actual output produced. Type the code yourself and experiment by changing values; that is the fastest way to learn.

1. Introduction to Object-Oriented Programming

Theory

Object-Oriented Programming (OOP) is a programming paradigm that organizes software around **objects** rather than functions and logic. An object bundles together **data** (called attributes or state) and the **behaviour** that operates on that data (called methods). Real-world things such as a student, a bank account or a car map naturally to objects, which makes large programs easier to design, read and maintain.

In **procedural programming**, data and functions live separately and functions pass data around. As a program grows, it becomes hard to track which function changes which data. OOP solves this by keeping the data and the functions that are allowed to change it inside the same unit: the class.

Python is a fully object-oriented language: *everything* in Python is an object, including integers, strings, functions, modules and even classes themselves. Every object has an identity (id), a type and a value.

Key Points

- **Class**: a blueprint or template that defines attributes and methods.
- **Object (Instance)**: a concrete thing created from a class, with its own data.
- **The four pillars of OOP**: Encapsulation, Abstraction, Inheritance and Polymorphism.
- **Benefits**: modularity, code reuse, easier maintenance, data security, and natural modelling of real-world problems.

Procedural Programming	Object-Oriented Programming
Program is divided into functions	Program is divided into objects
Top-down approach	Bottom-up approach
Data moves freely between functions	Data is hidden inside objects
Hard to reuse and extend	Reuse through inheritance & composition
Example: C	Example: Python, Java, C++

Practical Implementation

Program 1.1: Procedural vs Object-Oriented approach

CODE

```
# Procedural style: data and functions are separate
balance = 1000
def deposit(bal, amount):
    return bal + amount
balance = deposit(balance, 500)
print("Procedural balance:", balance)

# Object-oriented style: data + behaviour live together
class Account:
    def __init__(self, balance):
        self.balance = balance
    def deposit(self, amount):
        self.balance += amount
```

```
acc = Account(1000)
acc.deposit(500)
print("OOP balance:", acc.balance)
```

OUTPUT

```
Procedural balance: 1500
OOP balance: 1500
```

Program 1.2: Everything in Python is an object

CODE

```
x = 42
print(type(x), isinstance(x, object))
print(type("hello"), type(len), type(int))
print((5).bit_length())      # an int has methods too!
```

OUTPUT

```
<class 'int'> True
<class 'str'> <class 'builtin_function_or_method'> <class 'type'>
3
```

2. Classes and Objects

Theory

A **class** is defined with the `class` keyword. It is only a description: no memory is used for the data until we create an object. Creating an object from a class is called **instantiation**, and it is done by calling the class like a function: `obj = ClassName()`.

Each object has its own separate copy of instance attributes, so changing one object does not affect another. Attributes are accessed with the dot operator (`obj.attribute`) and methods are called the same way (`obj.method()`).

Key Points

- Class names use **PascalCase** by convention (e.g. `BankAccount`).
- Every object has a unique identity, which you can check with `id(obj)`.
- `isinstance(obj, Class)` checks whether an object belongs to a class.
- Attributes can even be added to an object at runtime, though it is better practice to define them in the constructor.

Practical Implementation

Program 2.1: Creating a class and multiple objects

CODE

```
class Car:
    """A simple Car class."""
    def start(self):
        print(f"{self.brand} {self.model} is starting...")

car1 = Car()
car1.brand, car1.model = "Tata", "Nexon"
car2 = Car()
car2.brand, car2.model = "Maruti", "Swift"

car1.start()
car2.start()
print("Same object?", car1 is car2)
print("car1 is a Car:", isinstance(car1, Car))
print("Docstring:", Car.__doc__)
```

OUTPUT

```
Tata Nexon is starting...
Maruti Swift is starting...
Same object? False
car1 is a Car: True
Docstring: A simple Car class.
```

3. Constructor (__init__) and the self Parameter

Theory

A **constructor** is a special method that runs automatically when an object is created. In Python the constructor is `__init__()`. It is used to initialise the object's attributes with starting values.

self is a reference to the current object. When you call `obj.method()`, Python internally calls `ClassName.method(obj)`, so the object is passed automatically as the first argument. The name *self* is a convention, not a keyword, but you should always use it.

Technically, object creation happens in two steps: `__new__()` creates (allocates) the object and `__init__()` initialises it. A **destructor**, `__del__()`, is called when the object is about to be destroyed by the garbage collector.

Key Points

- **Default constructor**: takes only self.
- **Parameterized constructor**: takes extra arguments to set attributes.
- Python does not support multiple constructors directly; use default arguments or class methods instead.
- __init__ must return None.

Practical Implementation

Program 3.1: Default and parameterized constructors

CODE

```
class Student:
    def __init__(self, name, roll, marks=0):   # parameterized with a default
        self.name = name
        self.roll = roll
        self.marks = marks
        print(f"Student object created for {name}")

    def display(self):
        print(f"Roll {self.roll}: {self.name} scored {self.marks}")

s1 = Student("Asha", 101, 92)
s2 = Student("Rahul", 102)          # marks uses default value
s1.display()
s2.display()
Student.display(s1)                 # what Python does internally
```

OUTPUT

```
Student object created for Asha
Student object created for Rahul
Roll 101: Asha scored 92
Roll 102: Rahul scored 0
Roll 101: Asha scored 92
```

Program 3.2: __new__, __init__ and __del__ in action

CODE

```
class Demo:
    def __new__(cls, *args):
        print("1. __new__ : creating object")
        return super().__new__(cls)
    def __init__(self, value):
        print("2. __init__: initialising with", value)
        self.value = value
    def __del__(self):
        print("3. __del__ : object destroyed")

d = Demo(10)
del d
```

OUTPUT

```
1. __new__ : creating object
2. __init__: initialising with 10
3. __del__ : object destroyed
```

4. Instance Variables vs Class Variables

Theory

Instance variables belong to a specific object. They are usually created inside `__init__` using `self`, and each object has its own copy.

Class variables (static variables) are defined directly inside the class body. They are shared by all objects of the class. They are useful for constants, counters and default settings that apply to every instance.

Be careful: assigning to a class variable through an object (`obj.var = x`) creates a new instance variable that *shadows* the class variable for that object only. To change it for everyone, assign through the class name.

Key Points

- Access class variables with `ClassName.var` (preferred) or `self.var`.
- `obj.__dict__` shows an object's instance variables.
- Never use a mutable object (list, dict) as a class variable unless you really want it shared.

Practical Implementation

Program 4.1: Shared counter and the shadowing trap

CODE

```
class Employee:
    company = "Infosys"      # class variable
    count = 0                # class variable

    def __init__(self, name, salary):
        self.name = name          # instance variable
        self.salary = salary      # instance variable
        Employee.count += 1

e1 = Employee("Priya", 50000)
e2 = Employee("Amit", 60000)
print("Total employees:", Employee.count)
print(e1.company, e2.company)

Employee.company = "TCS"          # changes for everyone
print(e1.company, e2.company)

e1.company = "Wipro"              # creates instance var on e1 only
print(e1.company, e2.company)
print("e1 dict:", e1.__dict__)
```

OUTPUT

```
Total employees: 2
Infosys Infosys
TCS TCS
Wipro TCS
e1 dict: {'name': 'Priya', 'salary': 50000, 'company': 'Wipro'}
```

5. Types of Methods: Instance, Class and Static

Theory

Python classes support three kinds of methods. **Instance methods** are the most common: they take `self` and can read or modify the object's state.

Class methods are marked with `@classmethod` and take `cls` (the class itself) as the first argument. They work with class-level data and are commonly used as **alternative constructors** (factory methods).

Static methods are marked with `@staticmethod` and take neither self nor cls. They are ordinary utility functions that logically belong to the class but do not need access to its state.

Feature	Instance Method	Class Method	Static Method
Decorator	none	@classmethod	@staticmethod
First argument	self	cls	none
Access object state	Yes	No	No
Access class state	Yes	Yes	No
Typical use	Object behaviour	Factory methods	Helper functions

Practical Implementation

Program 5.1: All three method types together

CODE

```
class Date:
    def __init__(self, day, month, year):
        self.day, self.month, self.year = day, month, year

    # Instance method
    def show(self):
        return f"{self.day:02d}/{self.month:02d}/{self.year}"

    # Class method -> alternative constructor
    @classmethod
    def from_string(cls, text):
        d, m, y = map(int, text.split("-"))
        return cls(d, m, y)

    # Static method -> utility
    @staticmethod
    def is_leap(year):
        return year % 4 == 0 and (year % 100 != 0 or year % 400 == 0)

d1 = Date(15, 8, 1947)
d2 = Date.from_string("26-01-1950")
print(d1.show(), "|", d2.show())
print("2024 leap?", Date.is_leap(2024))
print("1900 leap?", d1.is_leap(1900))
```

OUTPUT

```
15/08/1947 | 26/01/1950
2024 leap? True
1900 leap? False
```

6. Encapsulation and Access Modifiers

Theory

Encapsulation means wrapping data and the methods that work on it into a single unit (the class) and **restricting direct access** to some of the object's components. This protects the internal state from accidental or invalid changes; the outside world interacts only through a controlled public interface.

Unlike Java or C++, Python has no strict `private` keyword. Instead it uses naming conventions: a single underscore marks a member as **protected** (internal use, a hint to other programmers), and a double underscore makes it **private** by triggering **name mangling**, where `__balance` is renamed internally to `_ClassName__balance`.

Key Points

- **Public** (`name`): accessible from anywhere.
- **Protected** (`_name`): accessible, but should only be used inside the class and subclasses.
- **Private** (`__name`): name-mangled; not directly accessible from outside.
- Provide **getter** and **setter** methods to read and validate private data.

Practical Implementation

Program 6.1: Bank account with private balance

CODE

```
class BankAccount:
    def __init__(self, owner, balance):
        self.owner = owner            # public
        self._bank = "SBI"            # protected
        self.__balance = balance      # private

    def get_balance(self):            # getter
        return self.__balance

    def deposit(self, amount):        # controlled setter
        if amount <= 0:
            print("Invalid amount!")
            return
        self.__balance += amount

acc = BankAccount("Neha", 5000)
acc.deposit(2000)
acc.deposit(-50)
print("Balance:", acc.get_balance())
print("Protected:", acc._bank)

try:
    print(acc.__balance)
except AttributeError as e:
    print("Error:", e)

print("Name-mangled access:", acc._BankAccount__balance)
```

OUTPUT

```
Invalid amount!
Balance: 7000
Protected: SBI
Error: 'BankAccount' object has no attribute '__balance'
Name-mangled access: 7000
```

7. Properties: the Pythonic Getter and Setter

Theory

Writing `get_x()` and `set_x()` methods everywhere is verbose. Python's `@property` decorator lets you access a method as if it were a simple attribute, while still running validation code behind the scenes.

A property has three parts: the **getter** (`@property`), the **setter** (`@name.setter`) and the **deleter** (`@name.deleter`). A property with only a getter is **read-only**. Properties are also great for **computed attributes** that are calculated from other data.

Practical Implementation

Program 7.1: Validation, read-only and computed properties

CODE

```
class Temperature:
    def __init__(self, celsius):
        self.celsius = celsius        # goes through the setter

    @property
    def celsius(self):
        return self._celsius

    @celsius.setter
    def celsius(self, value):
        if value < -273.15:
            raise ValueError("Below absolute zero!")
        self._celsius = value

    @property
    def fahrenheit(self):             # computed, read-only
        return self._celsius * 9 / 5 + 32

t = Temperature(25)
print(t.celsius, "C =", t.fahrenheit, "F")
t.celsius = 100
print(t.celsius, "C =", t.fahrenheit, "F")
try:
    t.celsius = -300
except ValueError as e:
    print("Error:", e)
try:
    t.fahrenheit = 50
except AttributeError as e:
    print("Error: fahrenheit is read-only")
```

OUTPUT

```
25 C = 77.0 F
100 C = 212.0 F
Error: Below absolute zero!
Error: fahrenheit is read-only
```

8. Inheritance

Theory

Inheritance allows a new class (the **child**, derived or subclass) to acquire the attributes and methods of an existing class (the **parent**, base or superclass). It models an **IS-A** relationship (a Dog *is an* Animal) and promotes code reuse: common code is written once in the parent.

The child can use everything from the parent, **add** new attributes and methods, and **override** parent methods to change behaviour. In Python 3, every class implicitly inherits from `object`.

Key Points

- **Single**: one child inherits from one parent.
- **Multilevel**: a chain, e.g. Grandparent -> Parent -> Child.
- **Hierarchical**: several children inherit from one parent.
- **Multiple**: one child inherits from more than one parent.
- **Hybrid**: a combination of two or more of the above types.
- `issubclass(Child, Parent)` checks the relationship.

Practical Implementation

Program 8.1: Single inheritance

CODE

```
class Animal:
    def __init__(self, name):
        self.name = name
    def eat(self):
        print(f"{self.name} is eating")

class Dog(Animal):                  # Dog inherits Animal
    def bark(self):
        print(f"{self.name} says Woof!")

d = Dog("Bruno")
d.eat()       # inherited
d.bark()      # own method
print(issubclass(Dog, Animal), isinstance(d, Animal))
```

OUTPUT

```
Bruno is eating
Bruno says Woof!
True True
```

Program 8.2: Multilevel inheritance

CODE

```
class Vehicle:
    def wheels(self): return "has wheels"
class Car(Vehicle):
    def engine(self): return "has an engine"
class ElectricCar(Car):
    def battery(self): return "has a battery"

ev = ElectricCar()
print("EV", ev.wheels(), ",", ev.engine(), "and", ev.battery())
```

OUTPUT

```
EV has wheels , has an engine and has a battery
```

Program 8.3: Hierarchical inheritance

CODE

```
class Shape:
    def describe(self):
        print(f"I am a {type(self).__name__}")
class Circle(Shape): pass
class Square(Shape): pass
class Triangle(Shape): pass

for s in (Circle(), Square(), Triangle()):
    s.describe()
```

OUTPUT

```
I am a Circle
I am a Square
I am a Triangle
```

Program 8.4: Multiple and hybrid inheritance

CODE

```
class Father:
    def skills(self): return "Gardening"
class Mother:
    def cooking(self): return "Cooking"
class Child(Father, Mother):       # multiple inheritance
    def own(self): return "Coding"

c = Child()
print(c.skills(), "+", c.cooking(), "+", c.own())
```

OUTPUT

```
Gardening + Cooking + Coding
```

9. super() and Method Resolution Order (MRO)

Theory

The built-in `super()` returns a proxy object that lets a child class call methods of its parent. Its most common use is inside `__init__` so that the parent can initialise its own attributes before the child adds more. It also avoids hard-coding the parent's name.

When a class has multiple parents, Python must decide which method to use if several define the same name. It follows the **Method Resolution Order (MRO)**, computed with the **C3 linearization** algorithm. The MRO searches the child first, then parents from left to right, ensuring each class appears only once and always before its own parents. This solves the classic **Diamond Problem**.

You can inspect the MRO with `ClassName.__mro__` or `ClassName.mro()`.

Practical Implementation

Program 9.1: Using super() to extend the parent constructor

CODE

```
class Person:
    def __init__(self, name, age):
        self.name, self.age = name, age
    def info(self):
        return f"{self.name}, {self.age} yrs"

class Teacher(Person):
    def __init__(self, name, age, subject):
        super().__init__(name, age)      # reuse parent logic
        self.subject = subject
    def info(self):
        return super().info() + f", teaches {self.subject}"

t = Teacher("Mr. Kulkarni", 45, "Physics")
print(t.info())
```

OUTPUT

```
Mr. Kulkarni, 45 yrs, teaches Physics
```

Program 9.2: The Diamond Problem solved by MRO

CODE

```
class A:
    def greet(self): print("A.greet")
class B(A):
    def greet(self):
        print("B.greet"); super().greet()
class C(A):
    def greet(self):
        print("C.greet"); super().greet()
class D(B, C):
    def greet(self):
        print("D.greet"); super().greet()

D().greet()
print([cls.__name__ for cls in D.__mro__])
```

OUTPUT

```
D.greet
B.greet
C.greet
A.greet
['D', 'B', 'C', 'A', 'object']
```

10. Polymorphism

Theory

Polymorphism (Greek: "many forms") means that the same interface or operation can behave differently depending on the object it acts on. For example, the + operator adds numbers but joins strings, and `len()` works on strings, lists and dictionaries.

In OOP, polymorphism lets you write code that works with a general type and automatically uses the correct specialised behaviour at runtime. Python supports it through **method overriding**, **duck typing**, **operator overloading** and (emulated) **method overloading**.

Method overriding (runtime polymorphism): a child class provides its own version of a method already defined in the parent. **Duck typing**: "If it walks like a duck and quacks like a duck, it is a duck" - Python cares about whether an object *has* the needed method, not about its actual type.

Practical Implementation

Program 10.1: Built-in polymorphism

CODE

```
print(5 + 3, "|", "Py" + "thon", "|", [1, 2] + [3])
print(len("hello"), len([1, 2, 3]), len({"a": 1}))
```

OUTPUT

```
8 | Python | [1, 2, 3]
5 3 1
```

Program 10.2: Method overriding (runtime polymorphism)

CODE

```
class Payment:
    def pay(self, amount):
        print(f"Paying Rs.{amount}")

class UPI(Payment):
    def pay(self, amount):
        print(f"Paid Rs.{amount} via UPI")

class CreditCard(Payment):
    def pay(self, amount):
        print(f"Paid Rs.{amount} via Credit Card (+2% fee = Rs.{amount*1.02:.0f})")

for method in [Payment(), UPI(), CreditCard()]:
    method.pay(1000)              # same call, different behaviour
```

OUTPUT

```
Paying Rs.1000
Paid Rs.1000 via UPI
Paid Rs.1000 via Credit Card (+2% fee = Rs.1020)
```

Program 10.3: Duck typing - no inheritance needed

CODE

```
class Duck:
    def speak(self): return "Quack"
class Robot:
    def speak(self): return "Beep boop"
class Person:
    def speak(self): return "Hello"

def make_it_speak(thing):         # doesn't care about the type
    print(type(thing).__name__, "->", thing.speak())

for obj in (Duck(), Robot(), Person()):
    make_it_speak(obj)
```

OUTPUT

```
Duck -> Quack
Robot -> Beep boop
Person -> Hello
```

11. Method Overloading and Operator Overloading

Theory

Method overloading means having several methods with the same name but different parameters. Python does **not** support it directly: if you define a method twice, the second definition replaces the first. The same effect is achieved with **default arguments**, ***args / **kwargs**, or `functools.singledispatchmethod`, which picks an implementation based on the argument's type.

Operator overloading lets your own classes work with Python operators such as +, -, *, ==, < by defining special (dunder) methods like `__add__`, `__sub__`, `__mul__`, `__eq__` and `__lt__`. This is a form of compile-time style polymorphism and makes custom types feel like built-in ones.

Operator	Magic Method	Operator	Magic Method
+	__add__	==	__eq__
-	__sub__	!=	__ne__
*	__mul__	<	__lt__
/	__truediv__	>	__gt__
//	__floordiv__	<=	__le__
%	__mod__	>=	__ge__
**	__pow__	[ ]	__getitem__

Practical Implementation

Program 11.1: Method overloading with default args and *args

CODE

```
class Calculator:
    def add(self, a, b=0, c=0):
        return a + b + c
    def total(self, *nums):
        return sum(nums)

calc = Calculator()
print(calc.add(5), calc.add(5, 10), calc.add(5, 10, 15))
print(calc.total(1, 2, 3, 4, 5))
```

OUTPUT

```
5 15 30
15
```

Program 11.2: Type-based overloading with singledispatchmethod

CODE

```
from functools import singledispatchmethod

class Printer:
    @singledispatchmethod
    def show(self, arg):
        print("Unknown type:", arg)
    @show.register
    def _(self, arg: int):
        print("Integer:", arg * 2)
    @show.register
    def _(self, arg: str):
        print("String:", arg.upper())
    @show.register
    def _(self, arg: list):
        print("List of", len(arg), "items")

p = Printer()
p.show(21); p.show("python"); p.show([1, 2, 3]); p.show(3.5)
```

OUTPUT

```
Integer: 42
String: PYTHON
List of 3 items
Unknown type: 3.5
```

Program 11.3: Operator overloading with a Vector class

CODE

```
class Vector:
    def __init__(self, x, y):
        self.x, self.y = x, y
    def __add__(self, other):  return Vector(self.x + other.x, self.y + other.y)
    def __sub__(self, other):  return Vector(self.x - other.x, self.y - other.y)
    def __mul__(self, k):      return Vector(self.x * k, self.y * k)
    def __eq__(self, other):   return (self.x, self.y) == (other.x, other.y)
    def __lt__(self, other):   return abs(self) < abs(other)
    def __abs__(self):         return (self.x**2 + self.y**2) ** 0.5
    def __repr__(self):        return f"Vector({self.x}, {self.y})"

v1, v2 = Vector(2, 3), Vector(1, 1)
print(v1 + v2, v1 - v2, v1 * 3)
print(v1 == Vector(2, 3), v2 < v1, abs(Vector(3, 4)))
```

OUTPUT

```
Vector(3, 4) Vector(1, 2) Vector(6, 9)
True True 5.0
```

12. Abstraction and Abstract Base Classes

Theory

Abstraction means showing only the essential features of an object and hiding the implementation details. When you drive a car you use the steering wheel and pedals without knowing how the engine works internally. In code, abstraction defines *what* an object does, not *how* it does it.

Python implements abstraction with the `abc` module. A class that inherits from `ABC` and has at least one method decorated with `@abstractmethod` is an **abstract class**. It **cannot be instantiated**; it acts as a contract that forces every subclass to implement the abstract methods. An abstract class can also contain normal (concrete) methods that subclasses share.

An abstract class whose methods are all abstract behaves like an **interface** in Java. Python also offers `typing.Protocol` for structural (duck-typed) interfaces.

Key Points

- Abstraction hides complexity; encapsulation hides data. They work together.
- A subclass that fails to implement every abstract method is still abstract and cannot be instantiated.
- Abstract properties are created by stacking `@property` on top of `@abstractmethod`.

Practical Implementation

Program 12.1: Abstract class as a contract

CODE

```
from abc import ABC, abstractmethod
import math

class Shape(ABC):
    @abstractmethod
    def area(self): pass

    @abstractmethod
    def perimeter(self): pass

    def describe(self):                # concrete method, shared
        print(f"{type(self).__name__}: area={self.area():.2f}, "
              f"perimeter={self.perimeter():.2f}")

class Circle(Shape):
    def __init__(self, r): self.r = r
    def area(self): return math.pi * self.r ** 2
    def perimeter(self): return 2 * math.pi * self.r

class Rectangle(Shape):
    def __init__(self, l, w): self.l, self.w = l, w
    def area(self): return self.l * self.w
    def perimeter(self): return 2 * (self.l + self.w)

for s in (Circle(5), Rectangle(4, 6)):
    s.describe()

try:
    Shape()
except TypeError as e:
    print("Error:", e)
```

OUTPUT

```
Circle: area=78.54, perimeter=31.42
Rectangle: area=24.00, perimeter=20.00
Error: Can't instantiate abstract class Shape without an implementation for abstract
    methods 'area', 'perimeter'
```

Program 12.2: Incomplete subclass cannot be created

CODE

```
from abc import ABC, abstractmethod

class Database(ABC):
    @abstractmethod
    def connect(self): pass
    @abstractmethod
    def query(self, sql): pass

class MySQL(Database):
    def connect(self): print("Connected to MySQL")
    # query() is missing!

try:
    MySQL()
except TypeError as e:
    print("Error:", e)
```

OUTPUT

```
Error: Can't instantiate abstract class MySQL without an implementation for abstract
    method 'query'
```

13. Magic (Dunder) Methods

Theory

Magic methods, also called **dunder** (double underscore) methods, are special methods that Python calls automatically in response to built-in operations. They let your objects integrate with the language: be printed nicely, compared, measured with `len()`, indexed, iterated, called like functions and used in `with` statements.

`__str__` returns a user-friendly string (used by print and str), while `__repr__` returns an unambiguous developer-oriented string (used in the shell and in containers). If only `__repr__` is defined, it is used for both.

Category	Magic Methods
Creation / destruction	__new__, __init__, __del__
String representation	__str__, __repr__, __format__
Comparison	__eq__, __ne__, __lt__, __le__, __gt__, __ge__
Arithmetic	__add__, __sub__, __mul__, __truediv__, __pow__, __radd__
Container	__len__, __getitem__, __setitem__, __delitem__, __contains__
Iteration	__iter__, __next__
Callable / context	__call__, __enter__, __exit__
Attribute access	__getattr__, __setattr__, __delattr__
Hashing / truth	__hash__, __bool__

Practical Implementation

Program 13.1: A Playlist that behaves like a built-in container

CODE

```
class Playlist:
    def __init__(self, name, songs):
        self.name, self.songs = name, list(songs)
    def __str__(self):  return f"Playlist '{self.name}' ({len(self)} songs)"
    def __repr__(self): return f"Playlist({self.name!r}, {self.songs!r})"
    def __len__(self):  return len(self.songs)
    def __getitem__(self, i): return self.songs[i]
    def __contains__(self, song): return song in self.songs
    def __iter__(self): return iter(self.songs)
    def __call__(self): print(f"Now playing: {self.songs[0]}")
    def __bool__(self): return len(self.songs) > 0

p = Playlist("Chill", ["Song A", "Song B", "Song C"])
print(p)                # __str__
print(repr(p))          # __repr__
print(len(p), p[1], p[-1], p[0:2])
print("Song B" in p)    # __contains__
for song in p:          # __iter__
    print(" -", song)
```

```
p()                      # __call__
print(bool(Playlist("Empty", [])))
```

OUTPUT

```
Playlist 'Chill' (3 songs)
Playlist('Chill', ['Song A', 'Song B', 'Song C'])
3 Song B Song C ['Song A', 'Song B']
True
 - Song A
 - Song B
 - Song C
Now playing: Song A
False
```

14. Composition and Aggregation (HAS-A Relationship)

Theory

Inheritance models **IS-A**; **composition** and **aggregation** model **HAS-A**: an object contains or uses other objects. A Car *has an* Engine; a Department *has* Employees.

In **composition** (strong association), the contained object is created and owned by the container. Its lifetime depends on the container: if the Car is destroyed, its Engine goes with it. In **aggregation** (weak association), the contained object exists independently and is merely referenced. Employees continue to exist even if the Department is closed.

A common design guideline is **"favour composition over inheritance"**: composition creates looser coupling, and parts can be swapped easily without deep class hierarchies.

Practical Implementation

Program 14.1: Composition: Car owns its Engine

CODE

```
class Engine:
    def __init__(self, hp): self.hp = hp
    def start(self): return f"{self.hp}HP engine roaring"

class Car:
    def __init__(self, model, hp):
        self.model = model
        self.engine = Engine(hp)      # created inside -> composition
    def drive(self):
        print(f"{self.model}: {self.engine.start()}")

Car("Mahindra XUV700", 200).drive()
```

OUTPUT

```
Mahindra XUV700: 200HP engine roaring
```

Program 14.2: Aggregation: Department references existing Employees

CODE

```
class Employee:
    def __init__(self, name): self.name = name

class Department:
    def __init__(self, title, members):
        self.title = title
        self.members = members        # passed in -> aggregation
    def show(self):
        print(self.title, "->", [e.name for e in self.members])

e1, e2 = Employee("Kiran"), Employee("Sneha")
dept = Department("R&D", [e1, e2])
dept.show()
del dept
print("Employee still exists:", e1.name)
```

OUTPUT

```
R&D -> ['Kiran', 'Sneha']
Employee still exists: Kiran
```

15. Data Classes

Theory

Many classes exist mainly to store data, and writing `__init__`, `__repr__` and `__eq__` for each one is repetitive. The `@dataclass` decorator (Python 3.7+) generates these methods automatically from type-annotated class attributes.

Useful options include `order=True` (generates comparison methods), `frozen=True` (makes instances immutable and hashable), and `field(default_factory=list)` for safe mutable defaults. `__post_init__` runs after the generated constructor for extra validation.

Practical Implementation

Program 15.1: Dataclass with ordering, defaults and validation

CODE

```
from dataclasses import dataclass, field

@dataclass(order=True)
class Product:
    price: float
    name: str
    tags: list = field(default_factory=list)

    def __post_init__(self):
        if self.price < 0:
            raise ValueError("Price cannot be negative")

p1 = Product(499.0, "Mouse", ["electronics"])
p2 = Product(1299.0, "Keyboard")
print(p1)
print("Equal?", p1 == Product(499.0, "Mouse", ["electronics"]))
print("Cheapest:", min(p1, p2).name)

@dataclass(frozen=True)
class Point:
    x: int
    y: int

pt = Point(1, 2)
try:
    pt.x = 10
except Exception as e:
    print("Frozen:", type(e).__name__)
```

OUTPUT

```
Product(price=499.0, name='Mouse', tags=['electronics'])
Equal? True
Cheapest: Mouse
Frozen: FrozenInstanceError
```

16. Iterators, Generators and Context Managers with Classes

Theory

An **iterator** is an object that produces values one at a time. A class becomes iterable by implementing `__iter__()` (returning the iterator) and `__next__()` (returning the next value, or raising `StopIteration` when finished). This is exactly what a `for` loop uses behind the scenes.

A **context manager** controls setup and cleanup around a block of code using the `with` statement. A class becomes a context manager by implementing `__enter__()` and `__exit__()`. `__exit__` runs even if an exception occurs, making it ideal for files, database connections and locks.

Practical Implementation

Program 16.1: Custom iterator class

CODE

```
class Countdown:
    def __init__(self, start):
        self.current = start
    def __iter__(self):
        return self
    def __next__(self):
        if self.current <= 0:
            raise StopIteration
        self.current -= 1
        return self.current + 1

print(list(Countdown(5)))
for n in Countdown(3):
    print("T-minus", n)
```

OUTPUT

```
[5, 4, 3, 2, 1]
T-minus 3
T-minus 2
T-minus 1
```

Program 16.2: Custom context manager class

CODE

```
import time

class Timer:
    def __enter__(self):
        print("Timer started")
        self.start = time.perf_counter()
        return self
    def __exit__(self, exc_type, exc, tb):
        elapsed = time.perf_counter() - self.start
        print(f"Timer stopped (exception: {exc_type.__name__ if exc_type else None})")
        return True          # suppress the exception

with Timer():
    total = sum(range(100000))
    print("Sum computed:", total)
```

```
with Timer():
    1 / 0                    # handled by __exit__
print("Program continues normally")
```

OUTPUT

```
Timer started
Sum computed: 4999950000
Timer stopped (exception: None)
Timer started
Timer stopped (exception: ZeroDivisionError)
Program continues normally
```

17. Custom Exceptions

Theory

Exceptions in Python are themselves classes, arranged in an inheritance hierarchy rooted at `BaseException`. You create your own exception types by inheriting from `Exception` (or a more specific subclass). Custom exceptions make error handling clearer, because callers can catch exactly the error they care about, and you can attach extra data as attributes.

Building a small hierarchy (one base exception for your application and specific subclasses) lets callers catch either a specific error or all errors from your module with a single except clause.

Practical Implementation

Program 17.1: Exception hierarchy for a banking app

CODE

```
class BankError(Exception):
    """Base exception for the bank module."""

class InsufficientFunds(BankError):
    def __init__(self, balance, amount):
        self.balance, self.amount = balance, amount
        super().__init__(f"Need Rs.{amount}, but balance is Rs.{balance}")

class AccountLocked(BankError):
    pass

def withdraw(balance, amount, locked=False):
    if locked:
        raise AccountLocked("Account is locked")
    if amount > balance:
        raise InsufficientFunds(balance, amount)
    return balance - amount

for args in [(1000, 300), (1000, 5000), (1000, 100, True)]:
    try:
        print("New balance:", withdraw(*args))
    except InsufficientFunds as e:
        print("InsufficientFunds:", e, "| short by", e.amount - e.balance)
    except BankError as e:
        print(type(e).__name__ + ":", e)
```

OUTPUT

```
New balance: 700
InsufficientFunds: Need Rs.5000, but balance is Rs.1000 | short by 4000
AccountLocked: Account is locked
```

18. Advanced Topics: __slots__, Class Decorators and Metaclasses

Theory

__slots__: By default, each object stores its attributes in a dictionary (`__dict__`), which is flexible but uses extra memory. Declaring `__slots__` fixes the set of allowed attributes, saving memory and slightly speeding up attribute access. It also prevents accidental creation of new attributes.

Class decorators: a function that receives a class and returns a (possibly modified) class. `@dataclass` is one example. They are useful for registering classes or adding behaviour.

Metaclasses: since classes are objects, they too have a type, called a metaclass. The default metaclass is `type`. A custom metaclass (inheriting from `type`) controls how classes themselves are created. A classic use is the **Singleton** pattern, which ensures only one instance of a class exists.

Practical Implementation

Program 18.1: __slots__ restricts attributes

CODE

```
class Point:
    __slots__ = ("x", "y")
    def __init__(self, x, y):
        self.x, self.y = x, y

p = Point(3, 4)
print(p.x, p.y, hasattr(p, "__dict__"))
try:
    p.z = 5
except AttributeError as e:
    print("Error:", e)
```

OUTPUT

```
3 4 False
Error: 'Point' object has no attribute 'z'
```

Program 18.2: Class decorator and a Singleton metaclass

CODE

```
registry = {}
def register(cls):                        # class decorator
    registry[cls.__name__] = cls
    return cls

@register
class Plugin: pass
print("Registered:", list(registry))

class SingletonMeta(type):                # metaclass
    _instances = {}
    def __call__(cls, *args, **kwargs):
        if cls not in cls._instances:
            cls._instances[cls] = super().__call__(*args, **kwargs)
        return cls._instances[cls]

class Config(metaclass=SingletonMeta):
    def __init__(self):
```

```
        print("Loading config once...")

a, b = Config(), Config()
print("Same instance?", a is b)
print("type(Config) =", type(Config).__name__, "| type(int) =", type(int).__name__)
```

OUTPUT

```
Registered: ['Plugin']
Loading config once...
Same instance? True
type(Config) = SingletonMeta | type(int) = type
```

19. SOLID Design Principles

Theory

Knowing OOP syntax is only half the job; designing good classes is the other half. The **SOLID** principles, introduced by Robert C. Martin, are five guidelines for writing maintainable, flexible object-oriented code.

S - Single Responsibility: a class should have only one reason to change. **O - Open/Closed**: classes should be open for extension but closed for modification. **L - Liskov Substitution**: objects of a subclass should be usable anywhere the parent is expected, without breaking the program. **I - Interface Segregation**: prefer several small, specific interfaces over one large general one. **D - Dependency Inversion**: depend on abstractions, not on concrete classes.

Practical Implementation

Program 19.1: Open/Closed + Dependency Inversion in practice

CODE

```
from abc import ABC, abstractmethod

class Notifier(ABC):                    # abstraction
    @abstractmethod
    def send(self, msg): pass

class EmailNotifier(Notifier):
    def send(self, msg): print("Email:", msg)
class SMSNotifier(Notifier):
    def send(self, msg): print("SMS:", msg)
class WhatsAppNotifier(Notifier):       # new channel, no old code changed
    def send(self, msg): print("WhatsApp:", msg)

class OrderService:                     # single responsibility
    def __init__(self, notifier: Notifier):   # depends on abstraction
        self.notifier = notifier
    def place_order(self, item):
        print(f"Order placed for {item}")
        self.notifier.send(f"Your {item} is confirmed!")

for n in (EmailNotifier(), SMSNotifier(), WhatsAppNotifier()):
    OrderService(n).place_order("Laptop")
```

OUTPUT

```
Order placed for Laptop
Email: Your Laptop is confirmed!
Order placed for Laptop
SMS: Your Laptop is confirmed!
Order placed for Laptop
WhatsApp: Your Laptop is confirmed!
```

20. Mini Project: Library Management System

Theory

This project combines almost every concept covered in this guide: an **abstract base class** (LibraryItem), **inheritance** (Book, DVD), **polymorphism** (each item computes its own late fee), **encapsulation** with private data and **properties**, **class variables** and **class methods**, **magic methods**, **composition/aggregation** (Library holds items and members) and a **custom exception**.

Study how each class has one clear responsibility, and how the Library works with the abstract type LibraryItem rather than with specific subclasses. Try extending it with a Magazine class: you will find you do not need to change the Library class at all.

Practical Implementation

Program 20.1: Complete implementation

CODE

```
from abc import ABC, abstractmethod

class LibraryError(Exception): pass

class LibraryItem(ABC):
    _next_id = 1                                  # class variable
    def __init__(self, title):
        self.item_id = LibraryItem._next_id
        LibraryItem._next_id += 1
        self.title = title
        self.__available = True                   # private
    @property
    def available(self): return self.__available
    def checkout(self):
        if not self.__available:
            raise LibraryError(f"'{self.title}' is already issued")
        self.__available = False
    def return_item(self): self.__available = True
    @abstractmethod
    def late_fee(self, days): ...                 # polymorphic
    def __str__(self):
        status = "Available" if self.available else "Issued"
        return f"[{self.item_id}] {type(self).__name__}: {self.title} ({status})"

class Book(LibraryItem):
    def __init__(self, title, author):
        super().__init__(title); self.author = author
    def late_fee(self, days): return days * 2

class DVD(LibraryItem):
    def late_fee(self, days): return days * 5 + 10

class Member:
    MAX_ITEMS = 2
    def __init__(self, name): self.name, self.items = name, []
    @classmethod
    def premium(cls, name):
        m = cls(name); m.MAX_ITEMS = 5; return m

class Library:
    def __init__(self): self.catalog = {}
    def add(self, *items):
```

```
            for it in items: self.catalog[it.item_id] = it
        def issue(self, item_id, member):
            if len(member.items) >= member.MAX_ITEMS:
                raise LibraryError(f"{member.name} reached limit")
            item = self.catalog[item_id]
            item.checkout(); member.items.append(item)
            print(f"Issued '{item.title}' to {member.name}")
        def receive(self, item, member, days_late=0):
            item.return_item(); member.items.remove(item)
            print(f"Returned '{item.title}', late fee Rs.{item.late_fee(days_late)}")
        def __len__(self): return len(self.catalog)

    lib = Library()
    lib.add(Book("Wings of Fire", "A.P.J. Abdul Kalam"),
            Book("Python Crash Course", "Eric Matthes"), DVD("Interstellar"))
    ravi, meera = Member("Ravi"), Member.premium("Meera")
    print("Items in library:", len(lib))

    for action in [lambda: lib.issue(1, ravi), lambda: lib.issue(3, ravi),
                   lambda: lib.issue(2, ravi), lambda: lib.issue(1, meera)]:
        try: action()
        except LibraryError as e: print("Error:", e)

    for it in lib.catalog.values(): print(it)
    lib.receive(lib.catalog[1], ravi, days_late=3)
    lib.receive(lib.catalog[3], ravi, days_late=2)
```

OUTPUT

```
Items in library: 3
Issued 'Wings of Fire' to Ravi
Issued 'Interstellar' to Ravi
Error: Ravi reached limit
Error: 'Wings of Fire' is already issued
[1] Book: Wings of Fire (Issued)
[2] Book: Python Crash Course (Available)
[3] DVD: Interstellar (Issued)
Returned 'Wings of Fire', late fee Rs.6
Returned 'Interstellar', late fee Rs.20
```

21. Interview Questions and Answers

Q1. What are the four pillars of OOP?

Encapsulation (bundling data with methods and restricting access), Abstraction (exposing only essentials), Inheritance (reusing code through IS-A relationships) and Polymorphism (one interface, many forms).

Q2. What is the difference between a class and an object?

A class is a blueprint that defines attributes and methods; an object is a concrete instance of that class, occupying memory and holding its own data.

Q3. What is self in Python?

A reference to the current instance, passed automatically as the first argument of instance methods. It is a convention, not a reserved keyword.

Q4. Does Python support private variables?

Not strictly. A double underscore prefix triggers name mangling (__x becomes _ClassName__x), which discourages but does not fully prevent outside access.

Q5. Difference between @classmethod and @staticmethod?

A class method receives the class (cls) and can access or modify class state, often used as a factory. A static method receives neither self nor cls and acts as a plain utility function.

Q6. Does Python support method overloading?

Not directly; a later definition replaces an earlier one. It is emulated with default arguments, *args/**kwargs or functools.singledispatchmethod.

Q7. What is MRO?

Method Resolution Order: the order in which Python searches classes for a method, computed by C3 linearization. View it with ClassName.__mro__.

Q8. What is the difference between __str__ and __repr__?

__str__ gives a readable, user-friendly representation; __repr__ gives an unambiguous, developer-friendly one, ideally valid code to recreate the object.

Q9. Can an abstract class be instantiated?

No. A class inheriting from ABC with at least one unimplemented @abstractmethod raises TypeError on instantiation.

Q10. Composition vs Inheritance?

Inheritance is IS-A (Dog is an Animal); composition is HAS-A (Car has an Engine). Composition gives looser coupling and is usually preferred when a relationship is not a true IS-A.

Q11. What is duck typing?

Determining suitability by the presence of methods and attributes rather than the object's actual type.

Q12. What does super() do?

Returns a proxy that delegates method calls to the next class in the MRO, commonly used to call the parent's __init__.

22. Quick Reference Cheat Sheet

Concept	Python Syntax / Keyword
Define class	class Name:
Constructor	def __init__(self, ...):
Create object	obj = Name(args)
Inheritance	class Child(Parent):
Call parent	super().method()
Protected / Private	_var / __var
Property	@property, @x.setter
Class method	@classmethod def m(cls):
Static method	@staticmethod def m():
Abstract class	class A(ABC): @abstractmethod
Operator overloading	__add__, __eq__, __lt__ ...
String form	__str__, __repr__
Data class	@dataclass
Restrict attributes	__slots__ = ('a', 'b')
Check type	isinstance(), issubclass(), type()
Inspect MRO	Class.__mro__ / Class.mro()

Practice Exercises

• Create a **Rectangle** class with length and width, and methods for area and perimeter. Add a class method `square(side)` as an alternative constructor.

• Build an **Employee** hierarchy (Manager, Developer, Intern) where each overrides a `calculate_salary()` method. Process them all in one loop.

• Write a **Fraction** class that supports +, -, *, ==, < and prints as 3/4 using magic methods.

• Design an abstract **Vehicle** class with abstract methods start() and stop(), then implement Bike and Truck.

• Implement a **Stack** class with push, pop, peek, __len__ and __str__, raising a custom StackEmptyError when popping an empty stack.

• Extend the Library Management System with a Magazine class and a search-by-title feature.